

# Session 15: NumPy Tricks - Data Science Mentorship Program

## Introduction

This session focuses on "hidden" or less famous NumPy functions. While basic NumPy functions are well-known, these 20-25 specific functions are extremely powerful for data science workflows, making code cleaner and more efficient.

### 1. np.sort

#### Explanation:

This function is used to sort the elements of an array. While Python has a built-in `sorted()` function, `np.sort()` is preferred in Data Science because it returns a NumPy array instead of a Python list, preserving performance and compatibility.

#### Code:

```
import numpy as np

# Creating a 1D array with 15 random integers between 1 and 100
a = np.random.randint(1, 100, 15)
# Creating a 2D array (6 rows, 4 columns)
b = np.random.randint(1, 100, (6, 4))

# Sorting 1D array
sorted_a = np.sort(a)

# Sorting 2D array
# Axis 0 = Column-wise sorting
# Axis 1 = Row-wise sorting
column_sorted_b = np.sort(b, axis=0)
row_sorted_b = np.sort(b, axis=1)

# Descending order sort trick
desc_a = np.sort(a)[::-1]
```

#### Line-by-line Explanation:

1. `import numpy as np`: Importing the library.

2. `a = np.random.randint(1, 100, 15)`: Generates 15 random integers between 1 and 100.
3. `np.sort(a)`: Sorts the 1D array in ascending order.
4. `np.sort(b, axis=0)`: Sorts each column independently.
5. `np.sort(b, axis=1)`: Sorts each row independently.
6. `np.sort(a)[-1]`: Since NumPy doesn't have a `reverse=True` parameter in `sort`, we use slicing to reverse the sorted array for descending order.

#### Key Points:

- Default sorting algorithm is **QuickSort** (fastest), but you can change it to **MergeSort** using the `kind` parameter for stable sorting.
- Time complexity:  $O(N \log N)$ .

## 2. np.append

#### Explanation:

Appends values to the end of an array. This is similar to the Python list's `append` but requires the axis to be specified for 2D arrays.

#### Code:

```
# Appending to 1D array
a_append = np.append(a, 200)

# Appending a column to a 2D array
# Adding a column of ones
new_col = np.ones((b.shape[0], 1))
b_append = np.append(b, new_col, axis=1)
```

#### Line-by-line Explanation:

1. `np.append(a, 200)`: Adds the number 200 at the end of array `a`.
2. `np.ones((b.shape[0], 1))`: Creates a column of 1s matching the number of rows in `b`.
3. `np.append(b, new_col, axis=1)`: Joins the new column to the right side of array `b`.

#### Key Points:

- If `axis` is not provided, the array is flattened (converted to 1D) before appending.

## 3. np.concatenate

#### Explanation:

Joins a sequence of arrays along an existing axis. It is more flexible than `vstack` or `hstack`.

**Code:**

```
c = np.arange(6).reshape(2,3)
d = np.arange(6,12).reshape(2,3)

# Row-wise concatenation (Vertical)
res_v = np.concatenate((c, d), axis=0)

# Column-wise concatenation (Horizontal)
res_h = np.concatenate((c, d), axis=1)
```

**Line-by-line Explanation:**

1. `np.concatenate((c, d), axis=0)`: Places array d below array c.
2. `axis=1`: Places array d to the right of array c.

## 4. np.unique

**Explanation:**

Finds the unique elements of an array and returns them in a sorted array. This is extremely useful for identifying unique categories in a dataset.

**Code:**

```
e = np.array([1,2,1,2,3,4,5,5,6,1])
unique_vals = np.unique(e)
```

**Key Points:**

- Very common in data preprocessing to find unique user IDs or category names.

## 5. np.expand\_dims

**Explanation:**

Expands the shape of an array by inserting a new axis at a specified position.

**Code:**

```
# Original 1D shape (15,)
# Expanding to 2D (1, 15)
expanded_a = np.expand_dims(a, axis=0)
```

### Line-by-line Explanation:

1. `np.expand_dims(a, axis=0)`: Wraps the array in an extra set of brackets, turning it into a 2D "row vector".

### Why use this?

In Machine Learning (like Keras/TensorFlow), models often expect "batches" of data. If you have a single image (1D or 2D), the model might require a 3D or 4D shape (batch\_size, width, height, channels). `expand_dims` allows you to turn a single data point into a batch of 1.

## 6. np.where

### Explanation:

This is one of the most powerful functions. It acts like an if-else block for arrays. It can return indices where a condition is met or replace values based on a condition.

### Code:

```
# 1. Finding indices
indices = np.where(a > 50)

# 2. Replacing values (if condition, replace_with, else_stay_as)
# Replace all values > 50 with 0
new_a = np.where(a > 50, 0, a)

# Replace even numbers with 0
even_replace = np.where(a % 2 == 0, 0, a)
```

### Line-by-line Explanation:

1. `np.where(a > 50)`: Returns a tuple containing an array of index positions where the value is greater than 50.
2. `np.where(a > 50, 0, a)`: "If value is > 50, use 0; otherwise, keep the original value from a."

## 7. np.argmax & np.argmin

### Explanation:

Returns the **indices** of the maximum and minimum values along an axis.

### Code:

```
max_idx = np.argmax(a)
min_idx = np.argmin(a)
```

```
# In 2D
max_row_idx = np.argmax(b, axis=1)
```

### Key Points:

- Useful when you don't care what the max value is, but rather **where** it is located (e.g., which category had the highest probability).

## 8. np.cumsum & np.cumprod

### Explanation:

Computes the cumulative sum or product of elements along a given axis.

### Code:

```
# Cumulative Sum
# [1, 2, 3] -> [1, 1+2, 1+2+3] -> [1, 3, 6]
cumulative_sum = np.cumsum(a)
```

```
# Cumulative Product
cumulative_prod = np.cumprod(a)
```

### Real-world Example:

Used for calculating total runs in a cricket career match-by-match or calculating daily cumulative subscriber growth on YouTube.

## 9. np.percentile

### Explanation:

Used to find the  $n^{th}$  percentile of the data.

### Code:

```
# 100th percentile = Max value
max_val = np.percentile(a, 100)
# 0th percentile = Min value
min_val = np.percentile(a, 0)
# 50th percentile = Median
median_val = np.percentile(a, 50)
```

**Key Points:**

- Essential in statistics for understanding data distribution and detecting outliers.

## 10. np.histogram

**Explanation:**

Instead of plotting, this function calculates the frequency count of data within specific "bins".

**Code:**

```
# Finding frequency in bins: 0-10, 10-20, ... 100
counts, bins = np.histogram(a, bins=[0,10,20,30,40,50,60,70,80,90,100])
```

## 11. np.corrcoef

**Explanation:**

Calculates the Pearson product-moment correlation coefficient. It measures the strength of the linear relationship between two variables.

**Code:**

```
salary = np.array([20000, 30000, 40000])
experience = np.array([1, 2, 3])

correlation = np.corrcoef(salary, experience)
```

**Key Points:**

- Returns a matrix. The value at [0, 1] is the correlation between the two arrays.
- Range: -1 to +1.
- +1: Perfect positive correlation.
- 0: No correlation.
- -1: Perfect negative correlation.

## 12. np.isin

**Explanation:**

Checks if elements of one array are present in another. It's like a batch "membership" operator.

**Code:**

```
search_items = [10, 20, 30]
mask = np.isin(a, search_items)
# Get actual values found
found_values = a[mask]
```

## 13. np.flip

**Explanation:**

Reverses the order of elements in an array.

**Code:**

```
# Reverse 1D
flipped_a = np.flip(a)

# Reverse 2D rows or columns
flipped_b_rows = np.flip(b, axis=0)
```

## 14. np.put

**Explanation:**

Replaces specified elements of an array with given values. **Important:** This operation happens "in-place" (it modifies the original array).

**Code:**

```
# Replace elements at index 0 and 1 with 110 and 530
np.put(a, [0, 1], [110, 530])
```

## 15. np.delete

**Explanation:**

Deletes elements at specific indices.

**Code:**

```
# Delete item at index 0
```

```
new_a = np.delete(a, 0)
```

## 16. Set Operations

### Explanation:

NumPy provides functions to perform set operations on 1D arrays.

### Code:

```
m = np.array([1,2,3,4,5])
n = np.array([3,4,5,6,7])

# Union (All items)
np.union1d(m, n)

# Intersection (Common items)
np.intersect1d(m, n)

# Set Difference (In m but not in n)
np.setdiff1d(m, n)

# Symmetric Difference (In either but not both)
np.setxor1d(m, n)

# Membership check
np.in1d(m, [1, 10]) # Returns [True, False, False, False, False]
```

## 17. np.clip

### Explanation:

Limits the values in an array. Values smaller than 'min' become 'min', and values larger than 'max' become 'max'.

### Code:

```
# Keep all values between 25 and 75
clipped_a = np.clip(a, a_min=25, a_max=75)
```

### Line-by-line Explanation:

1. `np.clip(a, 25, 75)`: Every number in `a` less than 25 is turned into 25. Every number greater than 75 is turned into 75.

## Summary Table (Tips & Tricks)

| Function              | Shortcut/Trick                                           |
|-----------------------|----------------------------------------------------------|
| <code>np.sort</code>  | Use <code>[::-1]</code> for descending order.            |
| <code>np.where</code> | Use as a vectorized if-else for data cleaning.           |
| <code>axis=0</code>   | Think "Columns" in 2D NumPy operations.                  |
| <code>axis=1</code>   | Think "Rows" in 2D NumPy operations.                     |
| <code>np.put</code>   | Use for permanent, in-place editing of specific indices. |

### Final Message from Teacher:

"These functions are your tools. You don't need to memorize them all today, but you should know they exist. When you face a complex logic problem in Pandas or ML, remember: NumPy probably has a built-in function for it."